

Outline:

- Introduction to stream processing using Apache Kafka
- Working with operators in Kubernetes, creating Apache Kafka clusters
- Cluster setup and using and creating basic Apache Kafka primitives: Topics, Partitions, Replicas, and using the Kafka API

Study environment:

- Lab materials are present on Moodle
- We will use AWS Academy resources:
 - https://www.awsacademy.com/vforcesite/LMS_Login
- In our laboratory we will use :
 - aws-cli - <https://docs.aws.amazon.com/cli/latest/userguide/install-cliv2-linux.html>
 - kubectl - <https://kubernetes.io/docs/tasks/tools/>

When you execute all commands from the command line with the version argument, they should not exit with an error.

Before ending the class :

- Please combine the results of all tasks in one single text file and attach it to UPEL (at the end of our meeting)
- At the end of lab:
 - Click the Reset button on Vocareum labs (not End Lab)

Task -1 – Prepare and Verify your environment

First, using the AWS Console, create a cluster in Elastic Kubernetes Service (in search dialog type EKS)

Step 1
Step 2
Step 3
Step 4
Step 5
Step 6

Configure cluster
Specify networking
Configure observability
Select add-ons
Configure selected add-ons settings
Review and create

Configure cluster

Configuration options - new [Info](#)
Choose how you would like to configure the cluster.

☐ Quick configuration (with EKS Auto Mode) - new
Quickly create a cluster with production-grade default settings. The configuration uses EKS Auto Mode to automate infrastructure tasks like creating nodes and provisioning storage.

☒ Custom configuration
To change default settings prior to creation, choose this option. This configuration gives the option to use EKS Auto Mode and customize the cluster's configuration.

EKS Auto Mode - new [Info](#)
Choose if you would like to use EKS's Auto Mode.

☒ Use EKS Auto Mode
EKS automates routine cluster tasks for compute, storage, and networking. When a new pod can't fit onto existing nodes, EKS creates a new node. EKS combines cluster infrastructure managed by AWS with integrated Kubernetes capabilities to meet application compute needs. [View pricing](#)

► Included capabilities

Cluster configuration [Info](#)

Name
Enter a unique name for this cluster. This property cannot be changed after the cluster is created.

suu-cluster-1

The cluster name should begin with letter or digit and can have any of the following characters: the set of Unicode letters, digits, hyphens and underscores. Maximum length of 100.

Cluster IAM role [Info](#)
Select the Cluster IAM role to allow the Kubernetes control plane to manage AWS resources on your behalf. This cannot be changed after the cluster is created. To create a new custom role, follow the instructions in the [Amazon EKS User Guide](#)

LabRole [Create recommended role](#)

Kubernetes version settings

Kubernetes version [Info](#)
Select Kubernetes version for this cluster.

1.31

Upgrade policy [Info](#)
Choose one of the following options. You can switch the setting later while the standard support period is in effect.

☒ Standard
This option supports the Kubernetes version for 14 months after the release date. There is no additional cost. When standard support ends, your cluster will be auto upgraded to the next version.

☐ Extended
This option supports the Kubernetes version for 36 months after the release date. The extended support period has an additional hourly cost that begins after the standard support period ends. When extended support ends, your cluster will be auto upgraded to the next version.

- Step 1
Configure cluster
- Step 2
Specify networking
- Step 3
Configure observability
- Step 4
Select add-ons
- Step 5
Configure selected add-ons settings
- Step 6
Review and create

Specify networking

Networking [Info](#)

IP address family and service IP address range cannot be changed after cluster creation.

VPC [Info](#)

Select a VPC to use for your EKS cluster resources.

vpc-027fc650f81ac161a | Default [Refresh](#)

Subnets [Info](#)

Choose the subnets in your VPC where the control plane may place elastic network interfaces (ENIs) to facilitate communication with your cluster. To create a new subnet, go to the corresponding page in the [VPC console](#).

Select subnets [Refresh](#) [Clear selected subnets](#)

subnet-06eddb215982ca0285 [X](#) subnet-0d8dcd442ec3ddba7 [X](#)
us-east-1a 172.31.80.0/20 us-east-1b 172.31.16.0/20

Additional security groups - optional [Info](#)

EKS automatically creates a cluster security group on cluster creation to facilitate communication between worker nodes and control plane. Optionally, choose additional security groups to apply to the EKS-managed Elastic Network Interfaces that are created in your control plane subnets. To create a new security group, go to the corresponding page in the [VPC console](#).

Select security groups [Refresh](#)

Choose cluster IP address family [Info](#)

Specify the IP address type for pods and services in your cluster.

- ☒ IPv4
- ☐ IPv6

Configure Kubernetes service IP address block [Info](#)

- ☒ Specify the range from which cluster services will receive IP addresses.

Configure remote networks to enable hybrid nodes - new [Info](#)

EKS Hybrid Nodes enables you to use on-premises and edge infrastructure as nodes in EKS clusters.

- ☒ Specify the CIDR blocks for your on-premises environments that you will use for hybrid nodes.

Cluster endpoint access [Info](#)

Configure access to the Kubernetes API server endpoint.

- ☐ Public
The cluster endpoint is accessible from outside of your VPC. Worker node traffic will leave your VPC to connect to the endpoint.
- ☒ Public and private
The cluster endpoint is accessible from outside of your VPC. Worker node traffic to the endpoint will stay within your VPC.
- ☐ Private
The cluster endpoint is only accessible through your VPC. Worker node traffic to the endpoint will stay within your VPC.

Advanced settings

[Cancel](#) [Previous](#) [Next](#)

Once it is done, add nodes to cluster. In the cluster options add nodes to Node group.

[EKS](#) > [Clusters](#) > suu-cluster

[suu-cluster](#)
[Refresh](#)
[Delete cluster](#)

Cluster info [Info](#)

Status

Kubernetes version [Info](#)

Support type

Provider

Active

1.29

Standard support until March 23, 2025

EKS

[Overview](#)
[Resources](#)
[Compute](#)
[Networking](#)
[Add-ons](#)
[Access](#)
[Observability](#)
[Upgrade insights](#)
[Update history](#)
[Tags](#)

Nodes (0) [Info](#)

Node name	Instance type	Node group	Created	Status
No Nodes				
This cluster does not have any Nodes, or you don't have permission to view them.				

Node groups (0) [Info](#)
[Edit](#)
[Delete](#)
[Add node group](#)

Group name	Desired size	AMI release version	Launch template	Status
No node groups				
This cluster does not have any node groups.				
Nodes that are not part of an Amazon EKS managed node group are not shown in the AWS console.				
Add node group				

- Step 1
● **Configure node group**
- Step 2
○ Set compute and scaling configuration
- Step 3
○ Specify networking
- Step 4
○ Review and create

Configure node group [info](#)

A node group is a group of EC2 instances that supply compute capacity to your Amazon EKS cluster. You can add multiple node groups to your cluster.

Node group configuration

These properties cannot be changed after the node group is created.

Name

Assign a unique name for this node group.

The node group name should begin with letter or digit and can have any of the following characters: the set of Unicode letters, digits, hyphens and underscores. Maximum length of 63.

Node IAM role [info](#)

Select the IAM role that will be used by the nodes. To create a new role, go to the [IAM console](#).



[Create recommended role](#)

❗ The selected role must not be used by a self-managed node group as this could lead to a service interruption upon managed node group deletion.

[Learn more](#)

Launch template [info](#)

These properties cannot be changed after the node group is created.

☒ Use launch template

Configure this node group using an EC2 launch template.

- Step 1
● Configure node group
- Step 2
● **Set compute and scaling configuration**
- Step 3
○ Specify networking
- Step 4
○ Review and create

Set compute and scaling configuration

Node group compute configuration

These properties cannot be changed after the node group is created.

AMI type [info](#)

Select the EKS-optimized Amazon Machine Image for nodes.

Capacity type

Select the capacity purchase option for this node group.

Instance types [info](#)

Select instance types you prefer for this node group.

t3.medium vCPU: 2 vCPUs Memory: 4 GiB Network: Up to 5 Gigabit Max ENI: 3 Max IPs: 18

Disk size

Select the size of the attached EBS volume for each node.

 GiB

Next provision the cluster. First setup AWS account on your local machine, please perform following steps:

1. You can create environment by typing *aws configure*
 - a. Add at least one data to config
2. Copy your AWS credentials from Vocareum site to local machine. Store them in *~/.aws/credentials*.
3. Create default values for session:


```
aws configure set default.region us-east-1
aws configure set default.output table
```
4. Check the environment (there should be no ec2 instances and no errors on console):


```
aws sts get-caller-identity
```

Next, configure the kubectl command

```
aws eks describe-cluster --region us-east-1 --name <your cluster> --query 'cluster.status' --output text
aws eks --region us-east-1 update-kubeconfig --name <your cluster>
```

You should see the info that the cluster is ready. Now check the status of nodes:

```
kubectl get nodes
```

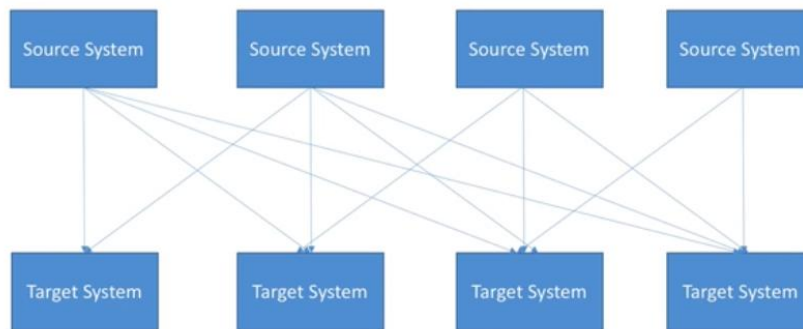
Also, check that all Kubernetes components are ready (here we are using *kube-system* namespace):

```
kubectl --namespace kube-system get pods
```

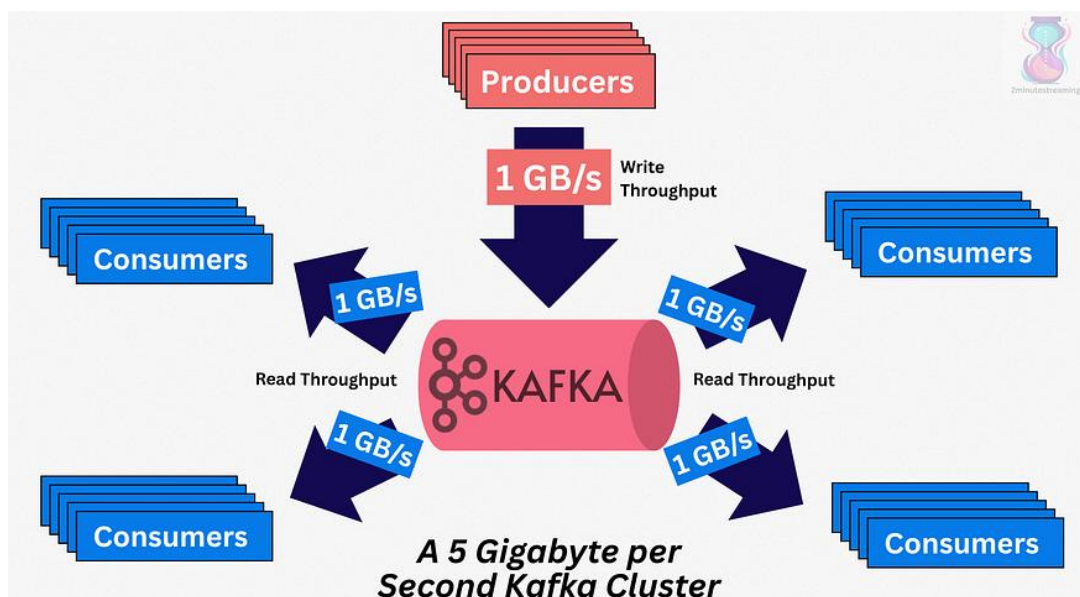
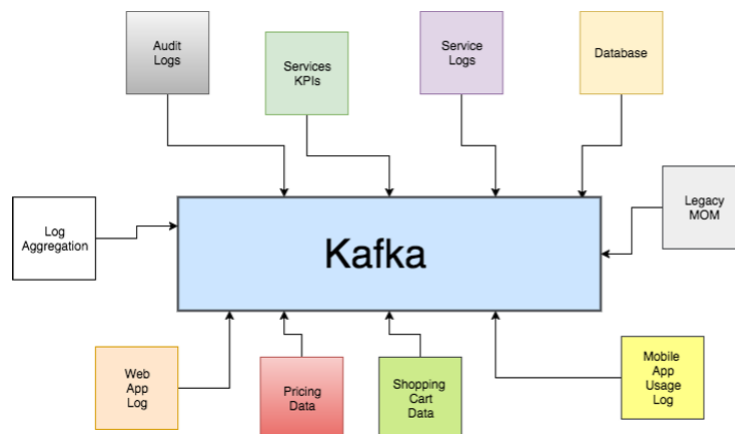
TASK -1 – list all nodes and components in your kube-system namespace

Task 0 - Introduction to Apache Kafka

In situations with many interacting systems, proper information exchange is difficult. The source systems require separate integration with the target system; there may be different protocols and data formats used for data exchange.



Apache Kafka comes in the middle and helps build data streams. In the intermediate layer, we decouple the streams.



The entire environment is distributed with resilient architecture and fault tolerance. It allows scaling horizontally. It also allows delivering messages in high performance, with latency less than 10 ms.

It is widely used in the industry as a *streaming platform*. “A Streaming Platform means a system that allows you to store and process a large volume of streams of data. For example, a company like Uber has millions of drivers constantly streaming their GPS coordinates to Uber’s backend systems. A streaming platform can scale to handle this data and process it in real time as it arrives. This helps Uber make use of the data (e.g, recompute the route, figure out where there is traffic, etc).”

LinkedIn was created and is currently maintained by Confluent.

Kafka: a Distributed Messaging System for Log Processing

Jay Kreps
LinkedIn Corp.
jkreps@linkedin.com

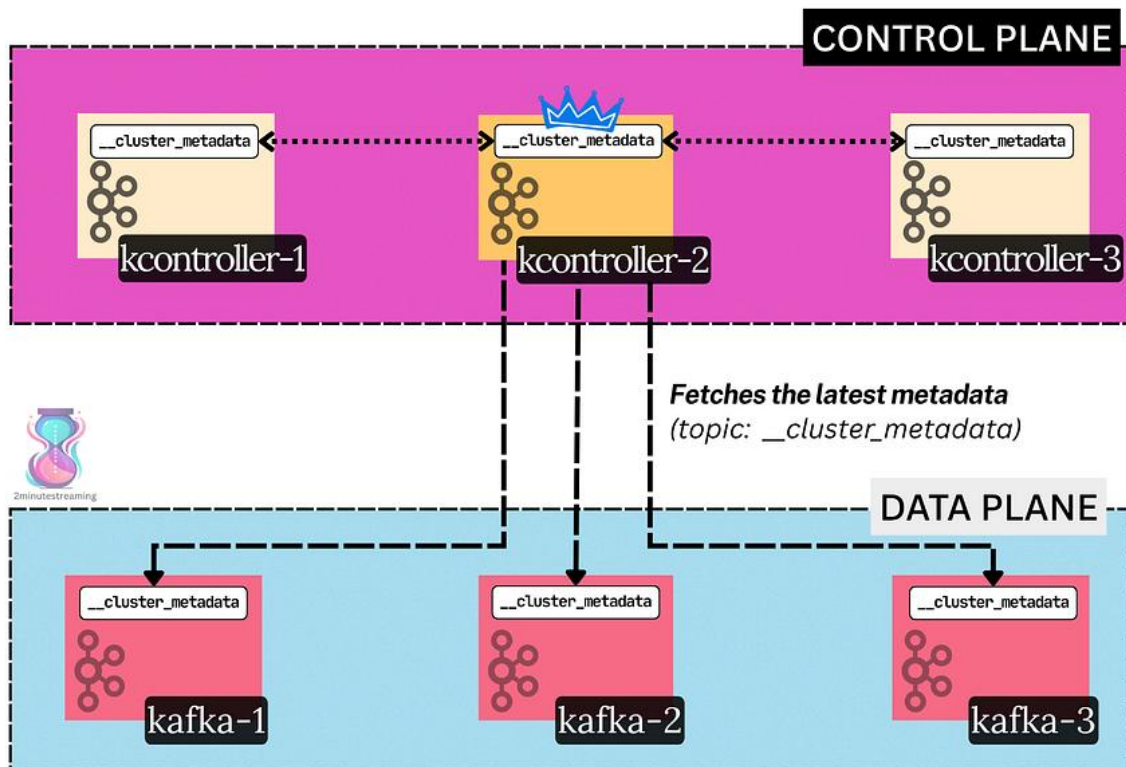
Neha Narkhede
LinkedIn Corp.
nnarkhede@linkedin.com

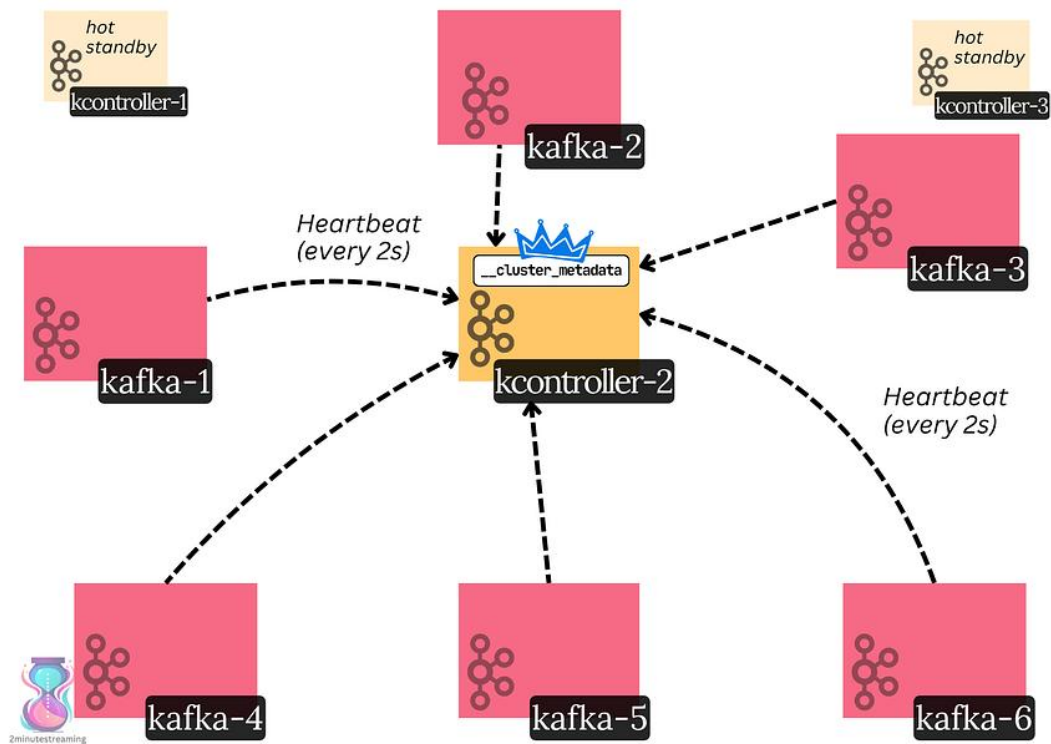
Jun Rao
LinkedIn Corp.
jrao@linkedin.com

The most popular use cases include:

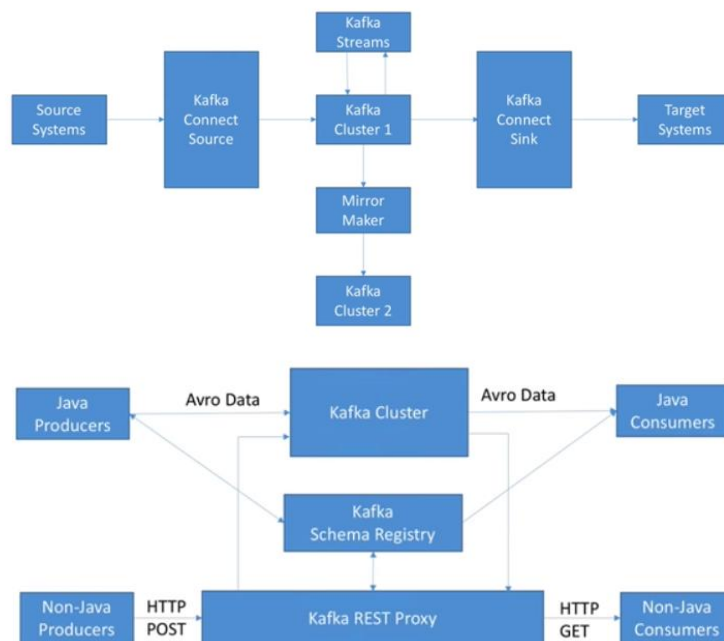
- messaging systems,
- users activity tracking,
- gather metrics from many different locations,
- gather application logs,
- stream processing (with Kafka Streams API or Spark),
- integration with Spark, Flink, Hadoop and other big data technologies.

The basic Kafka Core ecosystem consist of following components

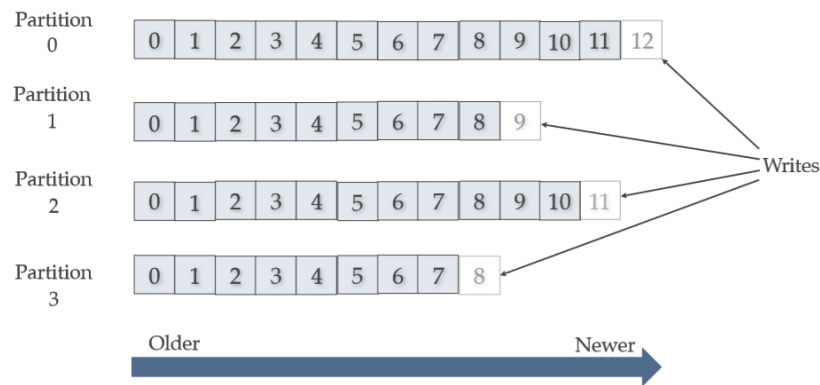




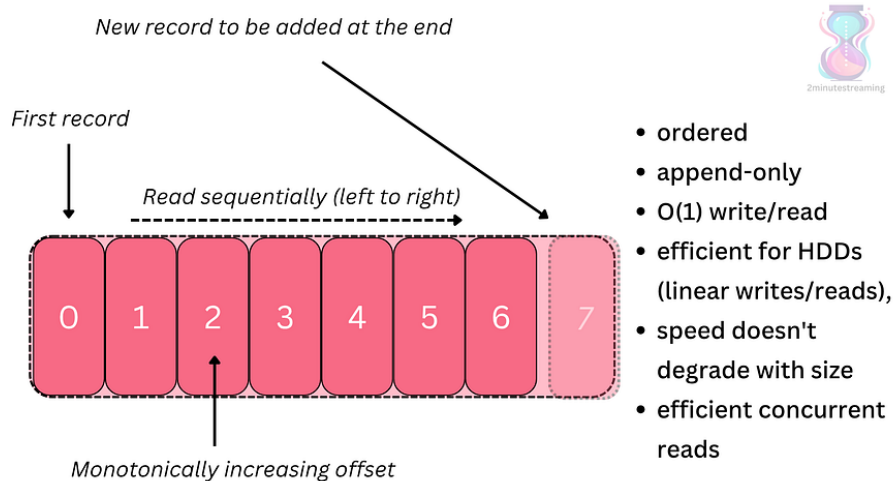
Confluent also release the Kafka Extended API, Schema Registry and REST Proxy



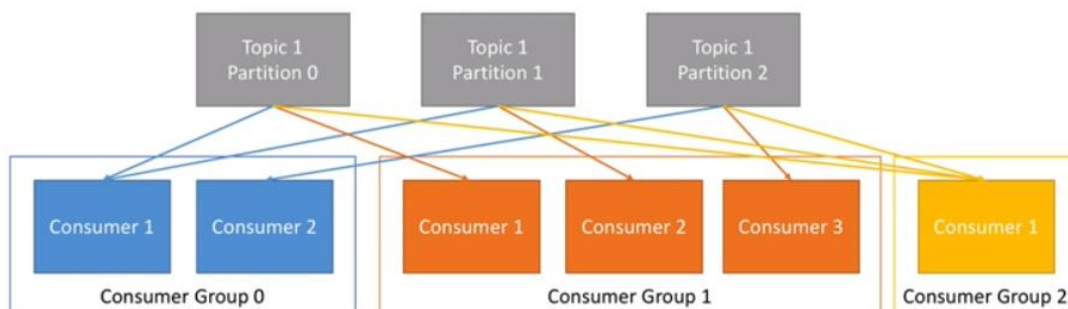
A topic is a stream of data (similar to a table in a database). Topics are split into partitions, and each partition is ordered. Each message within the partition gets an incremental ID, called an offset.



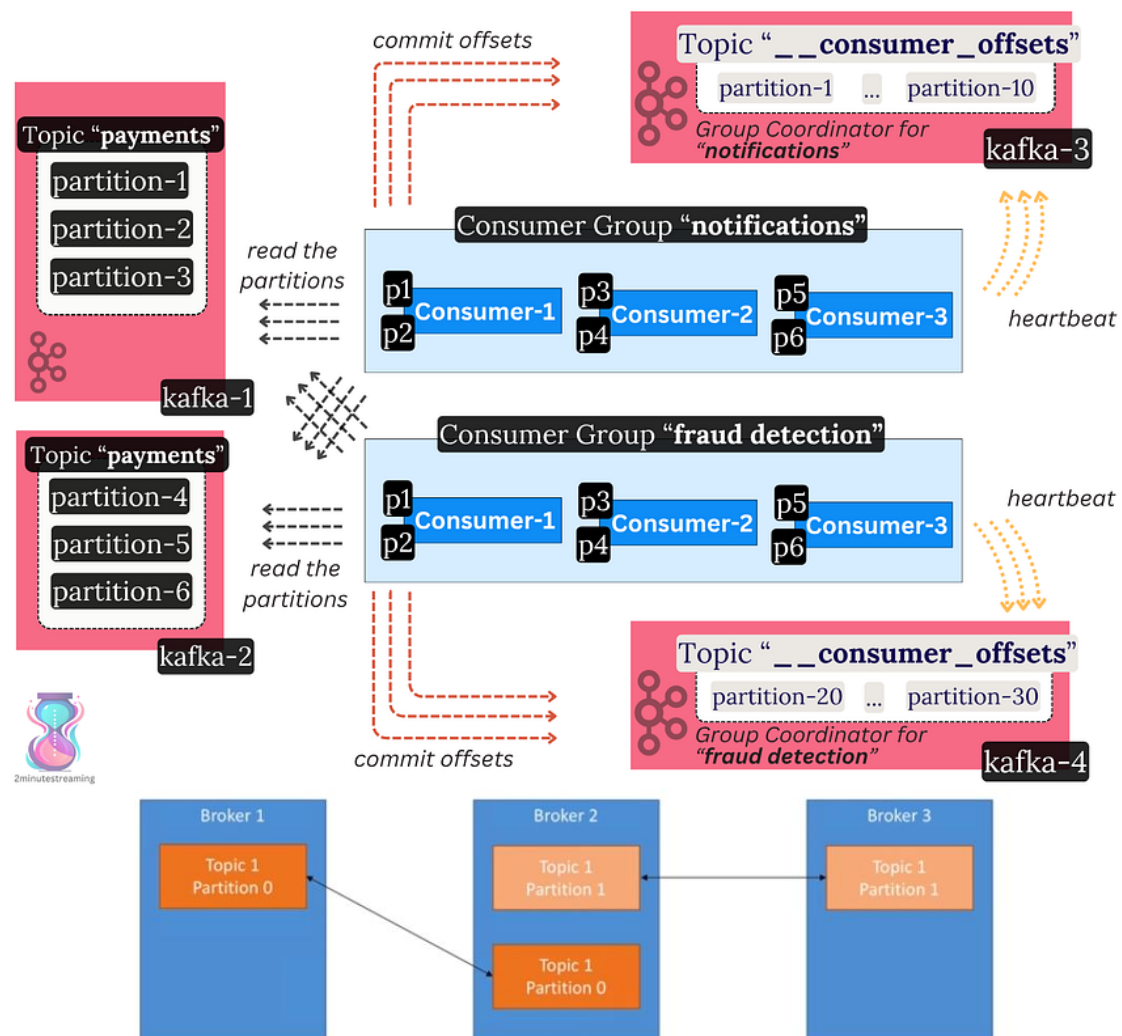
Offsets are valid only within a partition. Data is kept only for limited time and is immutable. An order is guaranteed only within a partition. Data is assigned randomly to a partition unless a key is provided.



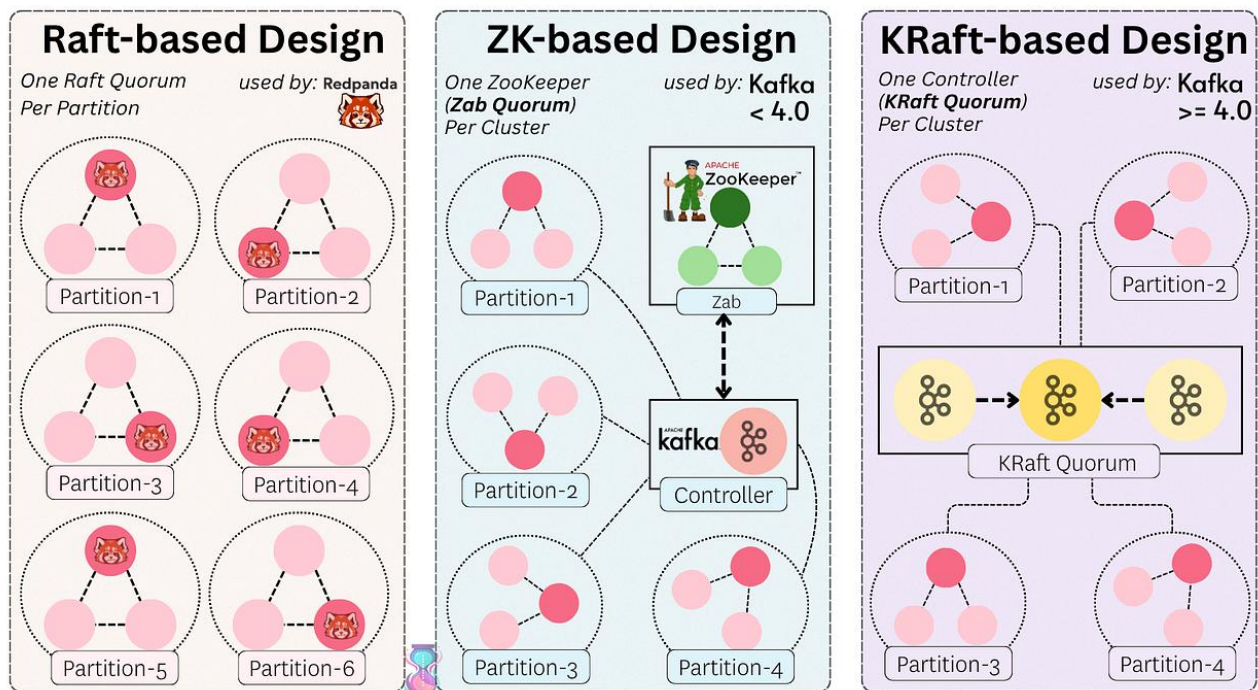
Consumer read data from a topic. They only have to specify the topic name and one broker to connect to, and Kafka will automatically take care of pulling the data from the right broker. Data is read in order for each partition. Consumers read data in consumer groups. Each consumer within a group reads from exclusive partitions. You cannot have more consumers than partitions.



Kafka stores the offsets at which a consumer group has been reading. The offsets commits are stored in a Kafka topic named "__consumer_offsets". If consumer process dies, it will be able to read back from where it left.



Kafka cluster is composed of multiple brokers, each of them is identified by unique ID. Topics have replication factors, usually greater than one. If the broker is down, another can serve the data. If one broker is down, we can still serve the data.



At any time only one broker can be a leader for a given partition. Only that leader can receive and serve data for a partition. The other brokers will synchronize the data. Each partition has one leader and multiple ISR (in-sync replica).

Task 0 – Strimzi Kafka Operator

First, let's discuss what an operator is. **Kubernetes Operator** is a program that runs inside a Kubernetes cluster and manages an application for you.

It watches Kubernetes resources, compares the **desired state** with the **actual state**, and takes actions to make the actual state match the desired state.

In simple terms:

```
You write: "I want Kafka with 3 brokers"
      ↓
Kubernetes stores this desired state
      ↓
The Operator observes it
      ↓
The Operator creates Pods, Services, configs, certificates, topics, users, etc.
      ↓
The Operator keeps checking and repairing the system
```

Why Operators exist - Kubernetes natively understands basic resources such as:

```
Pod
Deployment
Service
ConfigMap
Secret
PersistentVolumeClaim
```

But Kubernetes does **not** naturally understand complex applications like:

```
Kafka
PostgreSQL
Prometheus
Elasticsearch
Redis Cluster
```

These systems need domain-specific operations:

```
create a cluster
add/remove nodes
perform rolling upgrades
handle configuration changes
manage certificates
create users
rebalance replicas
recover from failures
```

An Operator encodes this operational knowledge as software. Operators usually introduce new Kubernetes object types called **Custom Resources**.

For Strimzi, examples are:

```
Kafka
KafkaNodePool
KafkaTopic
KafkaUser
KafkaConnect
KafkaMirrorMaker2
```

So instead of manually creating many Deployments, Services and ConfigMaps, we write something like:

```
apiVersion: kafka.strimzi.io/v1
kind: Kafka
metadata:
  name: suu-kafka-cluster
spec:
  kafka:
    listeners:
      - name: plain
        port: 9092
        type: internal
        tls: false
```

This tells Kubernetes:

I want a Kafka cluster called *suu-kafka-cluster*.

But Kubernetes itself does not know how to create Kafka. The **Strimzi Operator** sees this object and performs the actual work. In your lab, Strimzi is the Kafka Operator.

It manages:

```
Kafka brokers/controllers
KafkaNodePools
Services for internal and external access
LoadBalancers for external listeners
KafkaTopic resources
Kafka users
Entity Operator
Topic Operator
User Operator
certificates and configuration
rolling updates
```

In our lab, when we create this:

```
apiVersion: kafka.strimzi.io/v1
kind: KafkaTopic
metadata:
  name: orders
  labels:
    strimzi.io/cluster: suu-kafka-cluster
spec:
  partitions: 3
  replicas: 3
```

The **Topic Operator** notices it and creates the real Kafka topic:

```
orders
3 partitions
replication factor 3
```

So the flow is:

```
KafkaTopic YAML
↓
Kubernetes API
↓
Strimzi Topic Operator
↓
Kafka Admin API
↓
Topic exists in Kafka
```

Managing the desired and actual states is the key idea.

Desired state:

What you declared in YAML.

Actual state:

What is really running in the cluster.

Example:

```
spec:
  replicas: 3
```

Desired state says:

I want 3 Kafka nodes.

If one Kafka pod is deleted:

```
kubectl delete pod suu-kafka-cluster-dual-role-1 -n suu-kafka
```

the actual state becomes:

Only 2 Kafka pods are running.

The Operator/Kubernetes control loop brings the system back toward:

3 Kafka pods running.

A Kubernetes Operator follows the Operator pattern, and it is:

A controller that extends Kubernetes with application-specific knowledge and manages complex software through declarative custom resources.

Why it is important - without an Operator, you would manually manage:

```
broker pods
broker IDs
configuration files
listeners
advertised addresses
Services
LoadBalancers
storage
rolling restarts
topic creation
users and certificates
```

With Strimzi, you declare:

```
Kafka cluster
Kafka node pool
Kafka topics
Kafka users
```

and the Operator does the low-level Kubernetes work.

Now, let's create a Strimzi-based cluster¹. First unpack the archived attached to Upel. Inside you will find the most important files.

First, create a namespace for your cluster and all resources:

¹ https://strimzi.io/docs/operators/in-development/deploying#deploy-intro_str

```
kubectl create namespace suu-kafka
```

```
sed -i 's/namespace: ./namespace: suu-kafka/' install/cluster-operator/*RoleBinding*.yaml
```

Next, create an operator:

```
kubectl create -f install/cluster-operator -n suu-kafka
```

And create a cluster

```
kubectl apply -n suu-kafka -f examples/kafka/kafka-with-dual-role-nodes.yaml
```

You can list all provisioned resources

```
kubectl get kafkanodepool -n suu-kafka  
kubectl get kafkatopic -n suu-kafka
```

TASK 0 – once the cluster is up, list all pods in the kafka cluster namespace

Task 1 – Creating topics

Next, create a sample host with Kafka utilities (kafka-client) and a sample web interface for your Kafka cluster (kafka-ui). Both files are in the suu-sample directory. Open web ui in browser.

Next, create a sample topic :

```
apiVersion: kafka.strimzi.io/v1  
kind: KafkaTopic  
metadata:  
  name: orders  
  labels:  
    strimzi.io/cluster: suu-kafka-cluster  
spec:  
  partitions: 3  
  replicas: 3  
  config:  
    retention.ms: 3600000
```

Next, using kafka-client, send and receive information from the topic:

```
#wyslanie wiadomosci:  
kubectl exec -n suu-kafka -it kafka-client -- \\\n  bin/kafka-console-producer.sh \\\n  --bootstrap-server suu-kafka-cluster-kafka-bootstrap:9092 \\\n  --topic orders  
  
#odebranie:  
kubectl exec -n suu-kafka -it kafka-client -- \\\n  bin/kafka-console-consumer.sh \\\n  --bootstrap-server suu-kafka-cluster-kafka-bootstrap:9092 \\\n  --topic orders \\\n  --from-beginning
```

Navigate to Ui and verify that the messages are inside it

Task 2 – Partitions and replication

Inside the binaries, there is also an interesting tool for measuring the performance of the clusters.

```
kubectrl exec -n suu-kafka -it kafka-client -- \
  bin/kafka-producer-perf-test.sh \
  --num-records 100000 \
  --record-size 10000 \
  --throughput -1 \
  --topic <topic-name> \
  --producer-props \
  bootstrap.servers=suu-kafka-cluster-kafka-bootstrap:9092
```

Next, create 3 topics

- 1 partition, 1 replica
- 10 partitions, 1 replica
- 10 partitions, 3 replicas

Measure the latency and throughput.

TASK 2 – add screenshot with the topics you created, attach a listing with the measurements

Task 3 – Kafka API

Using the confluent-python² library, create

Producer.py

- Connects to Strimzi load balancer (list all services and find the proper endpoint)
- Produces the orders and submit it to the topic *order* (3 replicas, 3 partitions)
- Define a list of people who can make an order and later generate a random number of orders for each of them
- As a key, set the ID of the person who made an order

Consumer.py

- Connects to Strimzi load balancer
- Consumes all the orders from topics

Observe how the messages are distributed among the partitions

TASK 3 – add source codes to the report

² <https://docs.confluent.io/kafka-clients/python/current/overview.html>